

Smart contract audit rain.interpreter

Content

Project description	3
Executive summary	3
Reviews	3
Technical analysis and findings	6
Security findings	8
**** Critical	8
*** High	8
H01 - Off-by-One in MAX_STACK_RHS_OFFSET Causes LHS Item Count Corruption	8
** Medium	9
M01 - Out-of-bounds Second-byte Read Causes Valid Decimals to Revert	9
M02 - Out-of-Bounds Memory Read and Garbage Literal Parsing in Pragma	10
M03 - Silent Truncation of Sub-Parser Dispatch Length	10
M04 - LHS Item Count Overflow Causes Bitwise Carry-Over and Parser State Corruption	11
M05 - Unbounded LHS Count in endLine() for Empty-RHS Lines	12
M06 - Semantic Manipulation and Implicit Validation via Malicious Extern Contracts	13
* Low	14
L01 - Uppercase Hexadecimal Prefix Bypasses Hex Parser and Fails Confusingly	14
L02 - Missing Bitwise Mask on Outputs in LibOpCall	15
L03 - Implicit Operand/Bytecode Synchronisation in LibOpCall	16
Informational	17
I01 - Dead Code: Malformed Hex Literal Error is Unreachable	17
I02 - Unused ParseState parameter in boundHex	17
I03 - Misleading Documentation Comment Regarding Non-ASCII Characters	18
I04 - Missing explicit constants index bounds check in LibOpExtern.integrity()	18
I05 - Architectural Fragility: Hardcoded InterpreterState Memory Layout	19
I06 - Float Identity Testing Relies on Implicit referenceFn() Divergence	20
I07 - Asymmetry Between integrity() and run() Bounds in Variable-Length Logic Opcodes	20
I08 - Missing Internal Enforcement of Memory Bounds in Parser	21
General Risks and Assumptions	23
Approach and methodology	24



Project description

This repository contains the concrete reference implementation of the Rainlang interpreter ecosystem, built upon the V4 interfaces. Unlike the interface library, this project provides the executable smart contracts that power the stack, including:

1. **Rainterpreter:** The core execution engine that evaluates Rainlang bytecode using a gas-efficient, function-pointer-based dispatch loop.
2. **RainterpreterParser:** An on-chain parser that compiles Rainlang source code directly into executable bytecode and constants.
3. **RainterpreterStore:** A storage implementation for managing persistent state variables across expressions.
4. **RainterpreterExpressionDeployer:** The factory contract responsible for orchestrating the parsing, integrity checking, and deployment of expressions.
5. **Standard Opcode Library:** A comprehensive library of standard operations (`LibAllStandardOps`) covering arithmetic, logic, ERC20/721 interactions, and system state access.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.interpreter

Reviews

Date	Repository	Commit
24/02/26	rain.interpreter	a4f5515bbddadb4299996b5d31b44aca2574bc9f
06/03/26	rain.interpreter	441e9b5b2326bea7413640e8b998d002f32989cb
07/03/26	rain.interpreter	13917ea1eb147b19bc4ce5e01fd10071cb63e1f0
23/03/26	rain.interpreter	b44c664fe7cc175748f8cf19a99cdb43d7f3a0d1



Scope

Contracts

src/abstract/BaseRainterpreterExtern.sol
src/abstract/BaseRainterpreterSubParser.sol
src/concrete/Rainterpreter.sol
src/concrete/RainterpreterDISPaiRegistry.sol
src/concrete/RainterpreterExpressionDeployer.sol
src/concrete/RainterpreterParser.sol
src/concrete/RainterpreterStore.sol
src/concrete/extern/RainterpreterReferenceExtern.sol
src/error/ErrBitwise.sol
src/error/ErrDeploy.sol
src/error/ErrEval.sol
src/error/ErrExtern.sol
src/error/ErrIntegrity.sol
src/error/ErrOpList.sol
src/error/ErrParse.sol
src/error/ErrStore.sol
src/error/ErrSubParse.sol
src/generated/Rainterpreter.pointers.sol
src/generated/RainterpreterExpressionDeployer.pointers.sol
src/generated/RainterpreterParser.pointers.sol
src/generated/RainterpreterReferenceExtern.pointers.sol
src/generated/RainterpreterStore.pointers.sol
src/lib/deploy/LibInterpreterDeploy.sol
src/lib/eval/LibEval.sol
src/lib/extern/LibExtern.sol
src/lib/extern/reference/literal/LibParseLiteralRepeat.sol
src/lib/extern/reference/op/LibExternOpContextCallingContract.sol
src/lib/extern/reference/op/LibExternOpContextRainlen.sol
src/lib/extern/reference/op/LibExternOpContextSender.sol
src/lib/extern/reference/op/LibExternOpIntInc.sol
src/lib/extern/reference/op/LibExternOpStackOperand.sol
src/lib/integrity/LibIntegrityCheck.sol
src/lib/op/00/LibOpConstant.sol
src/lib/op/00/LibOpContext.sol
src/lib/op/00/LibOpExtern.sol
src/lib/op/00/LibOpStack.sol
src/lib/op/LibAllStandardOps.sol
src/lib/op/bitwise/LibOpBitwiseAnd.sol
src/lib/op/bitwise/LibOpBitwiseOr.sol
src/lib/op/bitwise/LibOpCtPop.sol
src/lib/op/bitwise/LibOpDecodeBits.sol
src/lib/op/bitwise/LibOpEncodeBits.sol
src/lib/op/bitwise/LibOpShiftBitsLeft.sol
src/lib/op/bitwise/LibOpShiftBitsRight.sol



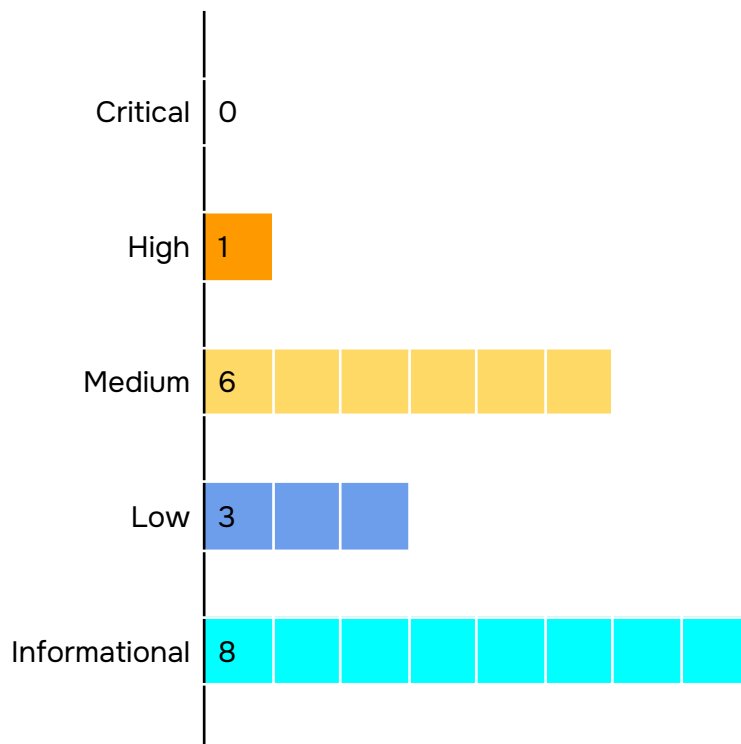
```
src/lib/op/call/LibOpCall.sol
src/lib/op/crypto/LibOpHash.sol
src/lib/op/erc20/LibOpERC20Allowance.sol
src/lib/op/erc20/LibOpERC20BalanceOf.sol
src/lib/op/erc20/LibOpERC20TotalSupply.sol
src/lib/op/erc20/uint256/LibOpUint256ERC20Allowance.sol
src/lib/op/erc20/uint256/LibOpUint256ERC20BalanceOf.sol
src/lib/op/erc20/uint256/LibOpUint256ERC20TotalSupply.sol
src/lib/op/erc5313/LibOpERC5313Owner.sol
src/lib/op/erc721/LibOpERC721BalanceOf.sol
src/lib/op/erc721/LibOpERC721OwnerOf.sol
src/lib/op/erc721/uint256/LibOpUint256ERC721BalanceOf.sol
src/lib/op/evm/LibOpBlockNumber.sol
src/lib/op/evm/LibOpChainId.sol
src/lib/op/evm/LibOpTimestamp.sol
src/lib/op/logic/LibOpAny.sol
src/lib/op/logic/LibOpBinaryEqualTo.sol
src/lib/op/logic/LibOpConditions.sol
src/lib/op/logic/LibOpEnsure.sol
src/lib/op/logic/LibOpEqualTo.sol
src/lib/op/logic/LibOpEvery.sol
src/lib/op/logic/LibOpGreaterThan.sol
src/lib/op/logic/LibOpGreaterThanOrEqualTo.sol
src/lib/op/logic/LibOpIf.sol
src/lib/op/logic/LibOpIsZero.sol
src/lib/op/logic/LibOpLessThan.sol
src/lib/op/logic/LibOpLessThanOrEqualTo.sol
src/lib/op/math/LibOpAbs.sol
src/lib/op/math/LibOpAdd.sol
src/lib/op/math/LibOpAvg.sol
src/lib/op/math/LibOpCeil.sol
src/lib/op/math/LibOpDiv.sol
src/lib/op/math/LibOpE.sol
src/lib/op/math/LibOpExp.sol
src/lib/op/math/LibOpExp2.sol
src/lib/op/math/LibOpFloor.sol
src/lib/op/math/LibOpFrac.sol
src/lib/op/math/LibOpGm.sol
src/lib/op/math/LibOpHeadroom.sol
src/lib/op/math/LibOpInv.sol
src/lib/op/math/LibOpMax.sol
src/lib/op/math/LibOpMaxNegativeValue.sol
src/lib/op/math/LibOpMaxPositiveValue.sol
src/lib/op/math/LibOpMin.sol
src/lib/op/math/LibOpMinNegativeValue.sol
src/lib/op/math/LibOpMinPositiveValue.sol
src/lib/op/math/LibOpMul.sol
src/lib/op/math/LibOpPow.sol
```



```
src/lib/op/math/LibOpSqrt.sol
src/lib/op/math/LibOpSub.sol
src/lib/op/math/growth/LibOpExponentialGrowth.sol
src/lib/op/math/growth/LibOpLinearGrowth.sol
src/lib/op/math/uint256/LibOpMaxUint256.sol
src/lib/op/math/uint256/LibOpUint256Add.sol
src/lib/op/math/uint256/LibOpUint256Div.sol
src/lib/op/math/uint256/LibOpUint256Mul.sol
src/lib/op/math/uint256/LibOpUint256Pow.sol
src/lib/op/math/uint256/LibOpUint256Sub.sol
src/lib/op/store/LibOpGet.sol
src/lib/op/store/LibOpSet.sol
src/lib/parse/LibParse.sol
src/lib/parse/LibParseError.sol
src/lib/parse/LibParseInterstitial.sol
src/lib/parse/LibParseOperand.sol
src/lib/parse/LibParsePragma.sol
src/lib/parse/LibParseStackName.sol
src/lib/parse/LibParseStackTracker.sol
src/lib/parse/LibParseState.sol
src/lib/parse/LibSubParse.sol
src/lib/parse/literal/LibParseLiteral.sol
src/lib/parse/literal/LibParseLiteralDecimal.sol
src/lib/parse/literal/LibParseLiteralHex.sol
src/lib/parse/literal/LibParseLiteralString.sol
src/lib/parse/literal/LibParseLiteralSubParseable.sol
src/lib/state/LibInterpreterState.sol
src/lib/state/LibInterpreterStateDataContract.sol
```



Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

H01 - Off-by-One in MAX_STACK_RHS_OFFSET Causes LHS Item Count Corruption

The ParseState struct compactly packs operation counters for Right-Hand Side (RHS) items into the 32-byte memory regions topLevel0 and topLevel1. Specifically, memory byte 0x5F (the final byte of topLevel1) is strictly reserved for the total Left-Hand Side (LHS) item counter.

To prevent the RHS operations counters from overflowing into this LHS counter, the system relies on the MAX_STACK_RHS_OFFSET constant. This constant is currently set to 0x3f (63).

When highwater() checks the RHS offset, it uses the condition if (newStackRHSOffset >= MAX_STACK_RHS_OFFSET). Because the constant is 63, the parser allows newStackRHSOffset to reach 62 safely. Consequently, a Rainlang expression is permitted to have exactly 63 top-level RHS items (indices 0 through 62).

The flaw emerges during pushOpToSource(). When processing the operations count for the 63rd RHS item (offset 62), the memory pointer is calculated as state + 0x20 + 62 + 1, which resolves exactly to offset 0x5F.

Because offset 0x5F is the physical memory byte where the LHS item counter is stored, every opcode pushed to the 63rd RHS item silently increments the total LHS item count instead of the RHS operations counter.

Impact:

This pointer collision silently corrupts the parser's core state machine. Artificially inflating the LHS item count, it destroys the stackLHSIndex assigned to named stack variables. Furthermore, it completely breaks the validation logic in endLine(), which relies on accurate LHS and RHS top-level counts to ensure stack safety. This can result in unpredictable parser crashes or, critically, trick the parser into successfully compiling logically corrupted bytecode that bypasses standard stack-depth safety rails.

Path: src/lib/parse/LibParseState.sol



Recommendation: Update the constant to safely reject offsets before they reach the LHS counter byte. The maximum safe offset is 61 (allowing 62 RHS items). Update the constant definition to uint256 constant MAX_STACK_RHS_OFFSET = 0x3e.

Found in: 13917ea1

Status: New

**** Medium**

M01 - Out-of-bounds Second-byte Read Causes Valid Decimals to Revert

When parsing literals, the system must differentiate between decimals and hexadecimals. If a literal begins with a numeric digit, `tryParseLiteral` immediately performs a look-ahead read for the next character in memory to check if it is the letter "x" (the "0x" prefix).

The flaw is that the parser performs this forward-read operation without verifying if the next character actually exists within the bounds of the input string.

If an input string ends precisely with a single decimal digit (e.g., a "0" at the very end of the buffer), the parser blindly fetches the next byte from out-of-bounds memory. In the EVM, memory is contiguous. If the uninitialized or adjacent memory byte coincidentally contains the ASCII value for "x" (0x78), the parser is tricked into believing the user wrote "0x".

The system incorrectly routes a perfectly valid, end-of-string decimal literal to the hexadecimal parser. The hex parser, observing that there are no actual hex digits within the valid bounds, crashes the transaction and reverts with a `ZeroLengthHexLiteral` error. This allows random EVM memory noise to cause a denial of service on valid developer inputs.

Path: `src/lib/parse/literal/LibParseLiteral.sol`

Recommendation: Add a strict boundary check before attempting to read the disambiguating second character. Verify that `cursor + 1 < end` before fetching the next byte. If the cursor is at the end of the string, safely default to routing to the decimal parser.

Found in: a4f5515b

Status: Fixed in 441e9b5b



M02 - Out-of-Bounds Memory Read and Garbage Literal Parsing in Pragma

The pragma parsing logic extracts sub-parser addresses using a while (cursor < end) loop. Inside this loop, it first calls `parseInterstitial` to skip any spaces, tabs, or comments before attempting to read the next address.

If an expression string ends precisely with trailing whitespace or a comment, `parseInterstitial` consumes those characters and advances the cursor until it exactly equals the end of the buffer. However, the loop lacks a boundary check immediately after this step. Instead of verifying if the input is exhausted, the program blindly proceeds to invoke `tryParseLiteral`.

Because `tryParseLiteral` is highly optimised, it does not perform its own bounds check. It unconditionally executes an `mload(cursor)` to inspect the starting character. Because `cursor == end`, this performs an out-of-bounds read, pulling in uninitialized memory or adjacent EVM data located past the source string.

If the adjacent, out-of-bounds memory coincidentally contains bytes matching the start of a valid literal (e.g., a quotation mark, bracket, or digit), the parser is tricked into processing garbage data. This can cause unpredictable denial-of-service reverts (like `UnclosedStringLiteral`) or, worse, successfully parse the garbage data and silently push a corrupted, non-existent sub-parser address into the active execution configuration.

Path: `src/lib/parse/LibParsePragma.sol`

Recommendation: Add a strict boundary check immediately after `parseInterstitial` completes inside the loop. If `cursor >= end`, safely break out of the loop before `tryParseLiteral` is ever executed.

Found in: `a4f5515b`

Status: Fixed in `441e9b5b`

M03 - Silent Truncation of Sub-Parser Dispatch Length

When preparing a bytes payload to send to a sub-parser, `subParseLiteral` tightly packs the data using low-level, overlapping memory writes.

First, it stores the `dispatchLength` shifted forward by 2 bytes: `mstore(add(data, 2), dispatchLength)`. Immediately after, it writes the total `dataLength` at the very start of the pointer: `mstore(data, dataLength)`.



Because `mstore` writes a full 32-byte word, the second write completely overwrites the upper 30 bytes of the previously written `dispatchLength`, preserving only the lowest 2 bytes (16 bits). This cleverly packs a 16-bit header. However, there is no boundary check on `dispatchLength` before this overlap occurs.

If a user provides a dispatch string larger than what 16 bits can hold (greater than `0xFFFF` or 65,535 bytes), the system silently erases the higher bits without throwing an error.

Because the length truncates silently, the main parser will send a corrupted payload. When the sub-parser unpacks the data, it will read the surviving 2 bytes and mistakenly believe the dispatch string is much shorter. This causes the sub-parser to misunderstand the structural boundary between the dispatch and body strings, leading to misclassified routing instructions, failed logic execution, and unpredictable smart contract behaviour.

Path: `src/lib/parse/LibSubParse.sol`

Recommendation: Add an explicit boundary check before packing the lengths. If `dispatchLength > 0xFFFF`, the process should immediately revert with a clear error. This ensures oversized inputs fail safely and loudly rather than silently corrupting the sub-parser payload.

Found in: `a4f5515b`

Status: Fixed in `441e9b5b`

M04 - LHS Item Count Overflow Causes Bitwise Carry-Over and Parser State Corruption

The `parseLHS()` function is responsible for parsing the Left-Hand Side of assignments (both named identifiers and anonymous `_` items). As it iterates through the items, it tracks the stack height by incrementing properties on the `ParseState` struct:

```
state.topLevel1++;  
state.lineTracker++;
```

Both `topLevel1` and `lineTracker` are tightly packed 256-bit words that store multiple independent 8-bit counters to optimise memory usage:

1. In `lineTracker`, the lowest byte (bits 0-7) tracks the current LHS item count, while the adjacent byte (bits 8-15) stores a critical snapshot of the RHS top-level word offset.



2. In `topLevel1`, the lowest byte tracks the total LHS stack height, and the adjacent byte stores the per-word operations depth for the 31st top-level RHS item.

Because the parser performs a blanket ++ operation directly on the 256-bit variables inside an unchecked block, it fails to isolate the increment to the lowest byte. If a Rainlang script provides exactly 256 items on the LHS of an assignment, the lowest byte will overflow from 0xFF to 0x00.

Crucially, standard binary arithmetic causes the carry bit to spill over into the adjacent byte (bits 8-15). This silently corrupts the parser's internal state machine:

1. The RHS offset snapshot in `lineTracker` is erroneously incremented. This corrupts the calculation of `lineRHSTopLevel` during `snapshotSourceHeadToLineTracker()` and `endLine()`, causing the parser to incorrectly shift memory pointers and read garbage memory to satisfy IO tracking.
2. The operations-depth counter in `topLevel1` is artificially inflated, which can cause out-of-bounds loops later in the parsing lifecycle.

This results in the parser losing synchronisation with the bytecode it is building, ultimately crashing with unpredictable and misleading errors (such as `LineRHSItemsOverflow`) rather than enforcing a clean, deliberate limit on stack depth.

Path: `src/lib/parse/LibParse.sol`

Recommendation: Enforce a strict upper bound of 255 items for the LHS stack depth by explicitly checking the lowest byte before incrementing the packed struct.

Found in: `a4f5515b`

Status: Fixed in `441e9b5b`

M05 - Unbounded LHS Count in `endLine()` for Empty-RHS Lines

In Rainlang, the `ParseState` struct tightly packs the operation counters for Right-Hand Side (RHS) items into `topLevel0` and `topLevel1` memory regions. To ensure these counters do not spill into other packed fields, the system uses a strict maximum limit defined by the `MAX_STACK_RHS_OFFSET` constant (63).

When parsing RHS items, `pushOpToSource()` correctly calls `highwater()`, which enforces this limit via the condition `newStackRHSOffset >= MAX_STACK_RHS_OFFSET`.

However, a bypass exists in `LibParse.sol` during `endLine()`. If an expression line contains only LHS items and no RHS expressions (e.g., `a b c ;`), the parser enters a special block where it blindly adds `lineLHSItems` directly to `totalRHSTopLevel`:



```
// As there are no RHS opcodes yet we can simply set topLevelO directly.  
totalRHSTopLevel += lineLHSItems;  
state.topLevelO = totalRHSTopLevel << 0xf8;
```

Because lineLHSItems can legitimately be up to 255, totalRHSTopLevel can easily be pushed far beyond the safe boundary of 63, breaking the parser's internal memory accounting invariant.

Impact:

This breaks the parser's internal memory accounting invariant. While the parser's rigid FSM and defensive `highwater()` gating prevent this from escalating into an arbitrary memory read or sandbox escape (as subsequent operations will read zeroed memory or hit the `highwater()` trap and revert), it guarantees a transaction revert during the parsing phase. This creates an unpredictable Denial of Service (DoS) vulnerability on expression deployment for valid syntax structures that happen to exceed the intended limit.

Path: `src/lib/parse/LibParseState.sol`

Recommendation: Add an explicit bounds check directly after `totalRHSTopLevel` is updated in `endLine()` to ensure it respects the same invariant that `highwater()` enforces. If the threshold is crossed, revert with `ParseStackOverflow()`.

Found in: 13917ea1

Status: New

M06 - Semantic Manipulation and Implicit Validation via Malicious Extern Contracts

In Rainlang, expression authors can reference arbitrary external contracts via the `constants` array. During compilation, `LibOpExtern.integrity()` delegates stack delta calculation to the external contract's `externIntegrity()` hook, blindly returning its stated input and output lengths without explicitly validating them against the `expectedInputsLength` and `expectedOutputsLength` encoded in the operand.

While the overarching `integrityCheck2` loop currently catches any structural mismatches (via a `BadOpInputsLength` revert), the opcode itself provides no local, explicit guarantee.

More concerning is the semantic threat: A malicious `InterpreterExternV4` contract can intentionally agree to the declared stack IO during the integrity check (returning the exact expected lengths), bypass all structural safety rails, and then execute arbitrary



logic to inject poisoned or crafted StackItem values into the execution environment during `extern()` at evaluation time.

Impact:

This poses two distinct risks. Structurally, the lack of local assertions in `LibOpExtern.integrity()` creates a latent fragility; if the central `integrityCheck2` is ever modified or bypassed, the interpreter would suffer stack drift. Semantically, it allows for data-layer manipulation. While memory bounds are maintained, the values pushed onto the stack are fully attacker-controlled. In multi-party protocols where one party provides an expression and another executes it, malicious externs serve as an active attack vector for manipulating state or outcomes.

Path: `src/lib/op/00/LibOpExtern.sol`

Recommendation: Make the integrity check self-validating so it does not rely solely on the generic outer loop. Capture the actual input and output lengths returned by the external contract's integrity hook, and explicitly assert that they exactly match the expected input and output lengths parsed from the operand. If there is a mismatch, revert with a dedicated error detailing the discrepancy. Additionally, explicitly document in the protocol's integration guidelines that extern contracts referenced in a Rainlang expression must be vetted and treated under the same trust model as the expression itself.

Found in: 13917ea1

Status: New

* Low

L01 - Uppercase Hexadecimal Prefix Bypasses Hex Parser and Fails Confusingly

When the interpreter reads a string of characters, it needs to quickly determine whether a number should be treated as a standard decimal or a hexadecimal value. To do this, it inspects the first two characters. If the first character is a zero, it checks the second character against a predefined mathematical mask to see if it is the letter "x".

The issue arises because this predefined mask is strictly configured to only recognise the lowercase letter "x". It does not include the uppercase letter "X", which is a widely accepted standard prefix for hexadecimal numbers in many programming environments.

If a user inputs a hexadecimal value using an uppercase prefix (for example, "0X1A"), the parser successfully identifies the leading zero as a numeric start but fails the check for



the lowercase "x". Because the prefix check fails, the routing logic incorrectly assumes the entire string must be a standard decimal float number.

The parser then routes the "OX1A" string directly into the decimal float parser. Naturally, the decimal parser does not know how to handle uppercase "X" or the following alphabetical characters, causing it to fail and throw a float-parsing error.

Impact:

This vulnerability does not pose a security risk because the invalid input is ultimately rejected by the system. However, it creates a significantly degraded and confusing developer experience.

Path: `src/lib/parse/literal/LibParseLiteral.sol`

Recommendation: The predefined mask used for dispatching hexadecimal literals should be updated to support both the lowercase "x" and the uppercase "X".

Found in: `a4f5515b`

Status: Fixed in `441e9b5b`

L02 - Missing Bitwise Mask on Outputs in LibOpCall

In `LibOpCall.sol`, the output length is extracted from the operand via an unmasked right shift:

```
uint256 outputs = uint256(OperandV2.unwrap(operand) >> 0x14);
```

The operand is designed to store the output's length strictly in bits 20-23 (a 4-bit value). However, by shifting right by `0x14` (20 bits) without an explicit bitwise AND mask (e.g., `& 0x0F`), the code technically reads the entire upper 244 bits of the operand. Sibling implementations, such as `LibOpExtern.sol`, correctly apply the `& 0x0F` mask after shifting.

Impact:

In the current architecture, this omission is harmless. `LibEval.evalLoop` preemptively masks the raw bytecode 32-byte word to 24 bits before passing it as an `OperandV2` to the opcode implementations. Because the operand is structurally capped at 24 bits, the bits above position 24 are guaranteed to be zero. However, this is a latent structural fragility. The opcode relies on an undocumented, implicit upstream truncation to prevent reading out-of-bounds output lengths. If `LibEval` is ever refactored or optimised to bypass this masking, `LibOpCall` would attempt to allocate massive amounts of stack memory, leading to a catastrophic memory underflow or crash.



Path: src/lib/op/call/LibOpCall.sol

Recommendation: Apply the standard 4-bit mask to the extraction logic to ensure defensive depth and consistency across opcodes. Update the extraction to uint256 outputs = uint256(OperandV2.unwrap(operand) >> 0x14) & 0x0F;

Found in: 13917ea1

Status: New

L03 - Implicit Operand/Bytecode Synchronisation in LibOpCall

In LibOpCall.sol, the integrity() function determines the required stack inputs by checking the target bytecode's metadata (sourceInputs) and simply returning it as the expected value. It completely ignores the inputs physically encoded within the user-provided operand.

The equality between the operand's encoded inputs and the actual sourceInputs is enforced purely by the generic BadOpInputsLength check in the overarching integrityCheck2 loop. This makes LibOpCall the only opcode where the integrity function's return value and the operand encoding are structurally independent quantities that happen to be constrained equal by an external safety net.

Impact:

This is a breach of the defence-in-depth principle. If the external integrityCheck2 logic is ever optimised, bypassed (such as in a fast-path evaluation), or removed, LibOpCall.integrity() would silently accept operands that request far more inputs than the target bytecode supports. The run() function would then pop the operand's requested amount, causing a severe stack underflow into unallocated memory.

Path: src/lib/op/call/LibOpCall.sol

Recommendation: Add an explicit equality assertion inside the LibOpCall integrity function to make the opcode self-documenting and structurally sealed. Extract the encoded inputs directly from the provided operand and compare them against the bytecode's natively expected source inputs. If the values are not strictly equal, revert the transaction with an appropriate input length error.

Found in: 13917ea1

Status: New



Informational

I01 - Dead Code: Malformed Hex Literal Error is Unreachable

Parsing hex literals involves two steps: bounding and parsing. First, `boundHex` scans the input using a strict `CMASK_HEX` bitmask, guaranteeing that only valid hex characters (0-9, a-f, A-F) are included within the determined boundaries.

Subsequently, `parseHex` loops through these bounded characters to convert them into a numerical value. It uses conditional branches to check if the character falls into the 0-9, a-f, or A-F ranges. It also includes a final `else` fallback branch that reverts with a `MalformedHexLiteral` error if a character fails all three checks.

Because the initial `boundHex` function strictly filters the input using the exact union of these three character ranges, it is mathematically impossible for a character to reach the final `else` branch. This makes the `MalformedHexLiteral` revert dead, unreachable code.

Path: `src/lib/parse/literal/LibParseLiteralHex.sol`

Recommendation: Either remove the unreachable `else` branch to save bytecode size, or retain it explicitly as a defensive fail-safe. If retained, add a comment clarifying that it is currently unreachable by design but exists to catch unhandled characters if the upstream `CMASK_HEX` filter is ever modified.

Found in: `a4f5515b`

Status: Acknowledged

I02 - Unused ParseState parameter in boundHex

The `boundHex` function inside `LibParseLiteralHex.sol` is responsible for scanning forward from a "0x" prefix to find the exact boundaries of a hexadecimal literal.

In its function signature, it declares an unnamed `ParseState` memory parameter. However, an inspection of the function's body reveals that this parameter is never accessed, read, or modified anywhere within the logic. Passing unused parameters bloats the function signature and slightly increases the cognitive load for developers trying to understand the data dependencies of the parsing step.

Path: `src/lib/parse/literal/LibParseLiteralHex.sol`



Recommendation: Remove the unused ParseState memory parameter from the boundHex function signature to clean up the code and accurately reflect the function's strict isolation from the overall parsing state.

Found in: a4f5515b

Status: Acknowledged

I03 - Misleading Documentation Comment Regarding Non-ASCII Characters

In parseSubParseable, a comment regarding the body parsing phase states that because the character mask is 128 bits, non-ASCII characters "MAY either fail to be skipped or will be treated as a closing bracket." This implies that the parser's behaviour is somewhat undefined or erratic when encountering non-ASCII text.

This is technically incorrect. Because bitwise operations in Yul/Solidity on masks like ~CMASK_SUB_PARSEABLE_LITERAL_END inherently operate on 256-bit numbers (where the upper 128 bits of the inverted mask become 1s), and because skipMask checks against this inverted mask, non-ASCII characters (which map to bits 128-255) will always cleanly stop the skipMask function.

Since the character stopping the mask will not be the valid closing bracket, the code will deterministically and safely revert with an UnclosedSubParseableLiteral error. The behaviour is strictly defined and safe, making the comment's warning of potential failure or misinterpretation misleading.

Path: src/lib/parse/literal/LibParseLiteralSubParseable.sol

Recommendation: Update the comment to accurately state that non-ASCII characters will unconditionally stop the parser and cleanly trigger an UnclosedSubParseableLiteral revert, removing the implication of undefined behaviour.

Found in: a4f5515b

Status: Fixed in 441e9b5b

I04 - Missing explicit constants index bounds check in LibOpExtern.integrity()

The *integrity()* function decodes the extern dispatch by first reading the constants entry at *encodedExternDispatchIndex*, derived from the low 16 bits of the operand.



Unlike *LibOpConstant.integrity()* and *LibOpStack.integrity()*, this function does not perform an explicit bounds check before indexing *state.constants[...]*. If the index is out of range due to malformed bytecode, parser/subparser inconsistency, or corrupted operand data, the access fails with a generic Solidity out-of-bounds panic instead of a domain-specific custom error. This weakens fail-fast diagnostics and makes triage/debugging less deterministic.

Path: `src/lib/op/00/LibOpExtern.sol`

Recommendation: Add an explicit bounds check in *LibOpExtern.integrity()* before reading *state.constants[encodedExternDispatchIndex]*, and revert with a dedicated custom error (or a shared existing out-of-bounds integrity error) that includes *opIndex*, constants length, and requested index.

Found in: b44c664f

Status: New

I05 - Architectural Fragility: Hardcoded InterpreterState Memory

Layout

LibOpStack.run accesses the *stackBottoms* array pointer using raw assembly by calling *mload(state)*.

Using *mload(state)* implicitly assumes that *stackBottoms* is declared as the very first field in the *InterpreterState* struct. While it currently works, if the struct is ever reordered (e.g., adding a new field before it), *LibOpStack* will silently load corrupt array pointers, leading to catastrophic runtime failure that the Solidity compiler will not catch.

Path: `src/lib/op/00/LibOpStack.sol`

Recommendation: Use the robust, explicit extraction pattern already utilised in *LibOpCall.sol*. Replace the implicit *mload(state)* with explicit routing. Declare a memory array for *stackBottoms* by reading it directly from *state.stackBottoms* in Solidity, and then perform the assembly *mload* operation on this newly declared array variable instead of the state pointer.

Found in: 13917ea1

Status: New



I06 - Float Identity Testing Relies on Implicit referenceFn() Divergence

In EVM block opcodes (LibOpBlockNumber, LibOpBlockTimestamp, LibOpChainId), the `run()` execution logic writes the raw EVM integer directly to the stack (e.g., `mstore(stackTop, number())`). In contrast, the `referenceFn()` implementation packages the value through `LibDecimalFloat.fromFixedDecimalLosslessPacked(value, 0)`.

A NatSpec comment justifies this divergence by stating that the reference function acts as a test to verify that `fromFixedDecimalLosslessPacked(value, 0)` is a bitwise identity function. While this mathematical property holds (the packed form of an integer with a 0 exponent is bitwise identical to the raw integer, provided it fits in `int224`), using the opcode's reference function to implicitly test the internal invariants of the Float library mixes architectural concerns.

Impact:

There is no direct security impact. However, if the Float library's packing logic is ever modified and the identity invariant breaks, the failure will surface indirectly as a confusing opcode discrepancy during differential testing, rather than a clean, isolated failure within the Float library's own test suite.

Path: `src/lib/op/evm/LibOpBlockNumber.sol`, `src/lib/op/evm/LibOpBlockTimestamp.sol`, `src/lib/op/evm/LibOpChainId.sol`

Recommendation: Introduce explicit fuzz/invariant tests within the Float library's dedicated test suite, asserting that `Float.unwrap(pack(x, 0)) == bytes32(x)` for all valid inputs. Once implemented, the comment in the EVM opcodes can be updated to simply state that the raw storage is a verified optimisation, decoupling the opcode from testing the math library's internal behaviour.

Found in: 13917ea1

Status: New

I07 - Asymmetry Between integrity() and run() Bounds in Variable-Length Logic Opcodes

The variable-length logic opcodes (LibOpAny, LibOpEvery, and LibOpConditions) explicitly clamp their minimum inputs to prevent zero-length processing. For example, in `LibOpAny.integrity()`, the inputs are clamped via `inputs = inputs > 0 ? inputs : 1`.

However, the corresponding `run()` functions calculate the stack lengths strictly from the raw operand bits without duplicating this clamp.



While the central `integrityCheck2` loop successfully prevents this mismatch from reaching execution via the `BadOpInputsLength` check, this design places the safety guarantee in a distant, generic location rather than co-locating it with the opcode logic.

Impact:

The asymmetry makes the code slightly harder to reason about during security reviews. More importantly, it creates a latent structural fragility. If the central integrity check were ever relaxed, a `run()` phase executing with zero inputs would naturally move the stack top downward without popping any items, inadvertently pushing uninitialized EVM memory as its output and causing stack drift.

Path: `src/lib/op/logic/LibOpAny.sol,` `src/lib/op/logic/LibOpEvery.sol,`
`src/lib/op/logic/LibOpConditions.sol`

Recommendation: Duplicate the minimum input clamping logic directly inside the `run` function of each affected opcode before calculating the stack length variable. This ensures the execution phase strictly respects the identical validation bounds, independent of any external checking loops.

Found in: 13917ea1

Status: New

I08 - Missing Internal Enforcement of Memory Bounds in Parser

In `LibParseState.sol`, the parsing system relies extensively on 16-bit relative memory pointers to track complex FSM state (e.g., `activeSourcePtr`, `subParsers`, `parenTracker`, `lineTracker`). Because these pointers are strictly 16 bits, they can safely address a maximum of 65,535 bytes (`0xFFFF`) of EVM memory. If the EVM free memory pointer exceeds this limit during a large parse operation, any newly stored pointers will silently truncate, leading to fundamentally corrupted linked lists and invalid bytecode generation.

To prevent this, the library provides the `checkParseMemoryOverflow()` function, which reads the EVM free memory pointer (`0x40`) and reverts if it reaches or exceeds `0x10000`. However, this critical safety check is never called internally by the core parsing entry point, `LibParse.parse()`. Instead, the library relies entirely on a `NatSpec` comment instructing the integrating caller contract to remember to enforce the check after the parsing routine returns.

Impact:

This design creates a severe architectural footgun. If an integrating developer fails to manually apply this check after invoking the parser, the system loses its primary



defence against memory-bound exhaustion. A highly complex or maliciously large Rainlang expression could force the internal memory pointers to silently truncate and wrap around, causing the parser to successfully compile logically corrupted, unpredictable bytecode without reverting the transaction.

Path: `src/lib/parse/LibParse.sol`, `src/lib/parse/LibParseState.sol`

Recommendation: Make the parser structurally secure and self-validating by enforcing the memory bounds check internally. Explicitly call the `state.checkParseMemoryOverflow()`; at the very end of the `LibParse.parse()` function, just before returning the assembled bytecode and constants array, removing the burden of memory-safety from the integrating contract.

Found in: 13917ea1

Status: New



General Risks and Assumptions

- **Bytecode Integrity Trust Model:** The Rainterpreter contract exposes `eval4()`, which deserialises and executes bytecode without re-running integrity checks for gas efficiency. It operates on the strict assumption that the bytecode was validated by `RainterpreterExpressionDeployer` (via `LibIntegrityCheck`) at deployment time. Calling `eval4()` with raw, unchecked, or manually constructed bytecode bypasses these safety rails and may result in undefined behaviour or memory corruption.
- **Function Pointer Alignment:** The system relies heavily on generated function pointer tables (`OPCODE_FUNCTION_POINTERS` and `INTEGRITY_FUNCTION_POINTERS`) for dynamic dispatch. The system assumes these tables are perfectly synchronised with the opcode indices defined in the Authoring Meta. Any misalignment between the compiled pointer tables and the opcode logic (e.g., due to a flawed build script or manual modification of generated files) will cause the interpreter to execute the wrong logic for a given opcode.
- **Sub-Parser Trust:** The parser supports the `using-words-from pragma`, which delegates parsing of unknown words to external sub-parser contracts. The system assumes these sub-parsers are trusted. A malicious or buggy sub-parser can return malformed bytecode that, while potentially caught by the subsequent integrity check, introduces a significant surface area for complexity and edge-case bugs during the parsing phase.
- **Context Authentication Responsibility:** While the interpreter provides read access to context data (e.g., via `LibOpContext`), it does not inherently validate the source or authenticity of that context. The system assumes the Calling Contract correctly authenticates signers and constructs the context grid, and that the Expression Logic enforces any necessary semantic constraints (deadlines, nonces, signatures) to prevent replay attacks.



Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of the implemented smart contracts and compare them with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.

